LEVEL 1 – PART 1 (Problems 1–25)

🔹 Problem 1: Create a Simple Class and Object

Concept: Class & Object

```
class Student {
  int id;
  String name;

  void display() {
    System.out.println(id + " " + name);
  }

  public static void main(String[] args) {
    Student s = new Student();
    s.id = 1;
    s.name = "Akash";
    s.display();
  }
}
```

Output

1 Akash

🔹 Problem 2: Multiple Objects of Same Class

```
class Employee {
  int id;
  String name;

  void show() {
```

```
        System.out.println(id + " " + name);
    }

    public static void main(String[] args) {
        Employee e1 = new Employee();
        Employee e2 = new Employee();

        e1.id = 101;
        e1.name = "John";

        e2.id = 102;
        e2.name = "David";

        e1.show();
        e2.show();
    }
}
```

🔹 **Problem 3: Default Constructor**

```
class Test {
    Test() {
        System.out.println("Default Constructor");
    }

    public static void main(String[] args) {
        Test t = new Test();
    }
}
```

🔹 Problem 4: Parameterized Constructor

```
class Person {
  int age;

  Person(int a) {
    age = a;
  }

  public static void main(String[] args) {
    Person p = new Person(25);
    System.out.println(p.age);
  }
}
```

🔹 Problem 5: Constructor Overloading

```
class Box {
  int length;

  Box() {
    length = 0;
  }

  Box(int l) {
    length = l;
  }

  public static void main(String[] args) {
```

```
    Box b1 = new Box();
    Box b2 = new Box(10);

    System.out.println(b1.length);
    System.out.println(b2.length);
  }
}
```

🔹 **Problem 6: Method Calling**

```
class Calculator {
  void add() {
    System.out.println(10 + 20);
  }

  public static void main(String[] args) {
    Calculator c = new Calculator();
    c.add();
  }
}
```

🔹 **Problem 7: Passing Parameters to Method**

```
class MathOp {
  void multiply(int a, int b) {
    System.out.println(a * b);
  }

  public static void main(String[] args) {
    MathOp m = new MathOp();
```

```
        m.multiply(5, 4);
    }
}
```

- **Problem 8: Returning Value from Method**

```
class Test {
    int square(int n) {
        return n * n;
    }

    public static void main(String[] args) {
        Test t = new Test();
        System.out.println(t.square(5));
    }
}
```

- **Problem 9: this Keyword – Variable Shadowing**

```
class Demo {
    int x;

    Demo(int x) {
        this.x = x;
    }

    public static void main(String[] args) {
        Demo d = new Demo(10);
        System.out.println(d.x);
    }
```

```
}
```

🔹 Problem 10: this to Call Method

```
class Sample {
  void show() {
    System.out.println("Show Method");
  }

  void display() {
    this.show();
  }

  public static void main(String[] args) {
    Sample s = new Sample();
    s.display();
  }
}
```

🔹 Problem 11: Static Variable

```
class Counter {
  static int count = 0;

  Counter() {
    count++;
  }

  public static void main(String[] args) {
    new Counter();
```

```
        new Counter();
        System.out.println(count);
    }
}
```

🔹 Problem 12: Static Method

```
class Utility {
    static void print() {
        System.out.println("Static Method");
    }

    public static void main(String[] args) {
        Utility.print();
    }
}
```

🔹 Problem 13: Static Block

```
class Test {
    static {
        System.out.println("Static Block");
    }

    public static void main(String[] args) {
        System.out.println("Main Method");
    }
}
```

🔹 Problem 14: Instance Initializer Block

```
class Demo {
  {
    System.out.println("Instance Block");
  }

  Demo() {
    System.out.println("Constructor");
  }

  public static void main(String[] args) {
    new Demo();
  }
}
```

🔹 Problem 15: final Variable

```
class Test {
  final int x = 10;

  public static void main(String[] args) {
    Test t = new Test();
    System.out.println(t.x);
  }
}
```

🔹 Problem 16: final Method

```
class A {
  final void show() {
    System.out.println("Final Method");
```

```
    }
}

class Test extends A {
    public static void main(String[] args) {
        new Test().show();
    }
}
```

🔹 Problem 17: final Class

```
final class A {
    void show() {
        System.out.println("Final Class");
    }

    public static void main(String[] args) {
        new A().show();
    }
}
```

🔹 Problem 18: Single Inheritance

```
class A {
    void showA() {
        System.out.println("Class A");
    }
}

class B extends A {
```

```
    public static void main(String[] args) {
        B b = new B();
        b.showA();
    }
}
```

- **Problem 19: Multilevel Inheritance**

```
class A {
    void a() {
        System.out.println("A");
    }
}

class B extends A {
    void b() {
        System.out.println("B");
    }
}

class C extends B {
    public static void main(String[] args) {
        C c = new C();
        c.a();
        c.b();
    }
}
```

- **Problem 20: Method Overriding**

```
class Parent {
  void show() {
    System.out.println("Parent");
  }
}

class Child extends Parent {
  void show() {
    System.out.println("Child");
  }

  public static void main(String[] args) {
    Parent p = new Child();
    p.show();
  }
}
```

🔹 **Problem 21: super Keyword – Method**

```
class A {
  void show() {
    System.out.println("A");
  }
}

class B extends A {
  void show() {
    super.show();
    System.out.println("B");
```

```
    }

    public static void main(String[] args) {
        new B().show();
    }
}
```

🔹 Problem 22: super Keyword – Constructor

```
class A {
    A() {
        System.out.println("A Constructor");
    }
}

class B extends A {
    B() {
        super();
        System.out.println("B Constructor");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

🔹 Problem 23: Method Overloading

```
class Test {
    void add(int a, int b) {
```

```
        System.out.println(a + b);
    }

    void add(double a, double b) {
        System.out.println(a + b);
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.add(10, 20);
        t.add(2.5, 3.5);
    }
}
```

🔹 **Problem 24: Interface Implementation**

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Circle");
    }

    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
    }
```

```
}
```

🔹 **Problem 25: Multiple Interfaces**

```
interface A {
  void showA();
}

interface B {
  void showB();
}

class Test implements A, B {
  public void showA() {
    System.out.println("A");
  }

  public void showB() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    Test t = new Test();
    t.showA();
    t.showB();
  }
}
```

Problem 26: Encapsulation using Getter & Setter

Concept: Encapsulation

```
class Student {
  private int marks;

  void setMarks(int m) {
    marks = m;
  }

  int getMarks() {
    return marks;
  }

  public static void main(String[] args) {
    Student s = new Student();
    s.setMarks(85);
    System.out.println(s.getMarks());
  }
}
```

🔹 **Problem 27: Encapsulation – Validation Logic**

```
class Account {
  private int balance;

  void setBalance(int b) {
    if (b >= 0)
      balance = b;
  }
```

```
  int getBalance() {
    return balance;
  }

  public static void main(String[] args) {
    Account a = new Account();
    a.setBalance(5000);
    System.out.println(a.getBalance());
  }
}
```

🔹 **Problem 28: Instance vs Local Variable**

```
class Test {
  int x = 10;

  void show() {
    int x = 20;
    System.out.println(x);
    System.out.println(this.x);
  }

  public static void main(String[] args) {
    new Test().show();
  }
}
```

Output

20

10

🔹 Problem 29: Static vs Instance Variable

```
class Demo {
  static int a = 10;
  int b = 20;

  public static void main(String[] args) {
    Demo d1 = new Demo();
    Demo d2 = new Demo();

    d1.a = 100;
    d2.b = 200;

    System.out.println(d1.a);
    System.out.println(d2.b);
  }
}
```

🔹 Problem 30: Static Method Accessing Static Data

```
class Sample {
  static int x = 50;

  static void show() {
    System.out.println(x);
  }

  public static void main(String[] args) {
    show();
```

```
    }
}
```

🔹 Problem 31: Instance Method Accessing Static Variable

```
class Test {
    static int x = 10;

    void display() {
        System.out.println(x);
    }

    public static void main(String[] args) {
        new Test().display();
    }
}
```

🔹 Problem 32: Static Method Cannot Access Instance Variable

```
class Demo {
    int x = 10;

    static void show() {
        // System.out.println(x); // compile-time error
    }

    public static void main(String[] args) {
        show();
    }
}
```

🔹 Problem 33: Object as Method Parameter

```
class Box {
  int size;
}

class Test {
  void show(Box b) {
    System.out.println(b.size);
  }

  public static void main(String[] args) {
    Box box = new Box();
    box.size = 15;

    Test t = new Test();
    t.show(box);
  }
}
```

🔹 Problem 34: Returning Object from Method

```
class Student {
  int id;

  Student create() {
    Student s = new Student();
    s.id = 101;
    return s;
```

```
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = s1.create();
        System.out.println(s2.id);
    }
}
```

🔹 **Problem 35: Constructor Calling Constructor (this())**

```
class Demo {
    Demo() {
        this(10);
        System.out.println("Default");
    }

    Demo(int x) {
        System.out.println(x);
    }

    public static void main(String[] args) {
        new Demo();
    }
}
```

🔹 **Problem 36: Order of Constructor Execution (Inheritance)**

```
class A {
    A() {
```

```
        System.out.println("A");
    }
}

class B extends A {
    B() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

🔹 **Problem 37: Method Overriding – Access Level**

```
class Parent {
    protected void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    public void show() {
        System.out.println("Child");
    }

    public static void main(String[] args) {
        Parent p = new Child();
```

```
        p.show();
    }
}
```

🔹 Problem 38: Method Overriding – Return Type

```
class A {
    Object show() {
        return "Hello";
    }
}

class B extends A {
    String show() {
        return "World";
    }

    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.show());
    }
}
```

🔹 Problem 39: Upcasting Example

```
class A {
    void show() {
        System.out.println("A");
    }
}
```

```
class B extends A {
  void display() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

🔹 **Problem 40: Downcasting (Safe Scenario)**

```
class A {
  void show() {
    System.out.println("A");
  }
}

class B extends A {
  void display() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    A a = new B();
    B b = (B) a;
    b.display();
```

```
    }
}
```

🔹 Problem 41: Runtime Polymorphism

```
class Shape {
    void draw() {
        System.out.println("Shape");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }

    public static void main(String[] args) {
        Shape s = new Circle();
        s.draw();
    }
}
```

🔹 Problem 42: instanceof Operator

```
class A {}
class B extends A {}

class Test {
    public static void main(String[] args) {
        A a = new B();
```

```
        System.out.println(a instanceof B);
    }
}
```

🔹 Problem 43: Abstract Class – Method Call

```
abstract class Vehicle {
    abstract void run();
}

class Bike extends Vehicle {
    void run() {
        System.out.println("Bike running");
    }

    public static void main(String[] args) {
        Vehicle v = new Bike();
        v.run();
    }
}
```

🔹 Problem 44: Abstract Class with Constructor

```
abstract class A {
    A() {
        System.out.println("Abstract Constructor");
    }
}

class B extends A {
```

```
  B() {
    System.out.println("Child Constructor");
  }

  public static void main(String[] args) {
    new B();
  }
}
```

🔹 **Problem 45: Interface Reference**

```
interface Animal {
  void sound();
}

class Dog implements Animal {
  public void sound() {
    System.out.println("Bark");
  }

  public static void main(String[] args) {
    Animal a = new Dog();
    a.sound();
  }
}
```

🔹 **Problem 46: Interface Variable (public static final)**

```
interface Test {
  int x = 10;
```

```
}

class Demo {
  public static void main(String[] args) {
    System.out.println(Test.x);
  }
}
```

🔹 **Problem 47: Interface Method Must be Implemented**

```
interface A {
  void show();
}

class B implements A {
  public void show() {
    System.out.println("Implemented");
  }

  public static void main(String[] args) {
    new B().show();
  }
}
```

🔹 **Problem 48: Multiple Inheritance via Interface**

```
interface A {
  void a();
}
```

```
interface B {
  void b();
}

class C implements A, B {
  public void a() {
    System.out.println("A");
  }

  public void b() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    C c = new C();
    c.a();
    c.b();
  }
}
```

🔹 **Problem 49: Method Overloading vs Overriding**

```
class A {
  void show(int x) {
    System.out.println("A " + x);
  }
}

class B extends A {
```

```
    void show(int x, int y) {
        System.out.println("B " + (x + y));
    }

    public static void main(String[] args) {
        B b = new B();
        b.show(10);
        b.show(10, 20);
    }
}
```

🔹 **Problem 50: Real-Life OOPS Modeling (Basic)**

```
class Car {
    void start() {
        System.out.println("Car started");
    }
}

class Test {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();
    }
}
```

Problem 51: Object Reference Assignment

```
class Test {
    int x = 10;
```

```
  public static void main(String[] args) {
    Test t1 = new Test();
    Test t2 = t1;

    t2.x = 50;
    System.out.println(t1.x);
  }
}
```

Concept: Reference sharing
Output: 50

🔹 Problem 52: Separate Objects, Separate Data

```
class Demo {
  int x = 10;

  public static void main(String[] args) {
    Demo d1 = new Demo();
    Demo d2 = new Demo();

    d1.x = 100;
    System.out.println(d2.x);
  }
}
```

Output: 10

🔹 Problem 53: Method Calling Order

```
class Test {
```

```
  void a() {
    System.out.println("A");
    b();
  }

  void b() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    new Test().a();
  }
}
```

🔹 **Problem 54: Constructor + Method Execution Order**

```
class Demo {
  Demo() {
    System.out.println("Constructor");
  }

  void show() {
    System.out.println("Method");
  }

  public static void main(String[] args) {
    Demo d = new Demo();
    d.show();
  }
```

```
}
```

🔹 Problem 55: Static Variable Shared Across Objects

```
class Counter {
  static int count = 0;

  Counter() {
    count++;
  }

  public static void main(String[] args) {
    new Counter();
    new Counter();
    new Counter();
    System.out.println(count);
  }
}
```

Output: 3

🔹 Problem 56: Instance Variable Per Object

```
class Demo {
  int x = 0;

  Demo() {
    x++;
  }

  public static void main(String[] args) {
```

```
        Demo d1 = new Demo();
        Demo d2 = new Demo();

        System.out.println(d1.x);
        System.out.println(d2.x);
    }
}
```

🔹 Problem 57: Static Method Calling Instance Method

```
class Test {
    void show() {
        System.out.println("Instance Method");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.show();
    }
}
```

🔹 Problem 58: Method Hiding (Static)

```
class A {
    static void show() {
        System.out.println("A");
    }
}

class B extends A {
```

```
    static void show() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Output: A
(**Static methods are not overridden**)

- **Problem 59: Constructor Overloading + this()**

```
class Box {
    int size;

    Box() {
        this(5);
    }

    Box(int s) {
        size = s;
    }

    public static void main(String[] args) {
        Box b = new Box();
        System.out.println(b.size);
    }
```

```
}
```

🔹 **Problem 60: Constructor Overloading + super()**

```
class A {
  A(int x) {
    System.out.println(x);
  }
}

class B extends A {
  B() {
    super(10);
  }

  public static void main(String[] args) {
    new B();
  }
}
```

🔹 **Problem 61: Overriding + super.method()**

```
class Parent {
  void show() {
    System.out.println("Parent");
  }
}

class Child extends Parent {
  void show() {
```

```
        super.show();
        System.out.println("Child");
    }

    public static void main(String[] args) {
        new Child().show();
    }
}
```

🔹 **Problem 62: Method Overloading with Type Promotion**

```
class Test {
    void add(int a, int b) {
        System.out.println("int");
    }

    void add(double a, double b) {
        System.out.println("double");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.add(5, 5);
        t.add(5.5, 6.5);
    }
}
```

🔹 **Problem 63: Overloading with Widening**

```
class Test {
```

```
  void show(long x) {
    System.out.println("long");
  }

  public static void main(String[] args) {
    Test t = new Test();
    t.show(10);
  }
}
```

🔹 Problem 64: Overloading with Char

```
class Test {
  void show(int x) {
    System.out.println("int");
  }

  public static void main(String[] args) {
    Test t = new Test();
    t.show('A');
  }
}
```

Output: int (char → int)

🔹 Problem 65: Abstract Method Implementation

```
abstract class A {
  abstract void show();
}
```

```
class B extends A {
  void show() {
    System.out.println("Implemented");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

- **Problem 66: Abstract Class Reference**

```
abstract class Shape {
  abstract void draw();
}

class Square extends Shape {
  void draw() {
    System.out.println("Square");
  }

  public static void main(String[] args) {
    Shape s = new Square();
    s.draw();
  }
}
```

- **Problem 67: Interface Reference Polymorphism**

```
interface Fly {
  void fly();
}

class Bird implements Fly {
  public void fly() {
    System.out.println("Flying");
  }

  public static void main(String[] args) {
    Fly f = new Bird();
    f.fly();
  }
}
```

- **Problem 68: Interface Variable is Constant**

```
interface Test {
  int x = 10;
}

class Demo {
  public static void main(String[] args) {
    // Test.x = 20; // compile-time error
    System.out.println(Test.x);
  }
}
```

- **Problem 69: Interface Method Overriding**

```
interface A {
  void show();
}

class B implements A {
  public void show() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

🔹 **Problem 70: Multiple Objects with Same Interface**

```
interface Shape {
  void draw();
}

class Circle implements Shape {
  public void draw() {
    System.out.println("Circle");
  }
}

class Square implements Shape {
  public void draw() {
```

```
        System.out.println("Square");
    }

    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Square();

        s1.draw();
        s2.draw();
    }
}
```

🔹 **Problem 71: Encapsulation with Constructor**

```
class Employee {
    private int salary;

    Employee(int s) {
        salary = s;
    }

    int getSalary() {
        return salary;
    }

    public static void main(String[] args) {
        Employee e = new Employee(50000);
        System.out.println(e.getSalary());
    }
```

```
}
```

🔹 Problem 72: Passing Object to Constructor

```
class Data {
  int x = 10;
}

class Test {
  Test(Data d) {
    System.out.println(d.x);
  }

  public static void main(String[] args) {
    Data d = new Data();
    new Test(d);
  }
}
```

🔹 Problem 73: Returning this from Method

```
class Test {
  Test get() {
    return this;
  }

  public static void main(String[] args) {
    Test t = new Test();
    System.out.println(t.get());
  }
```

```
}
```

🔹 Problem 74: this Used for Method Chaining

```
class Test {
  Test a() {
    System.out.println("A");
    return this;
  }

  Test b() {
    System.out.println("B");
    return this;
  }

  public static void main(String[] args) {
    new Test().a().b();
  }
}
```

🔹 Problem 75: Simple Real-Time OOPS Model

```
class Bank {
  void openAccount() {
    System.out.println("Account Opened");
  }
}

class Test {
  public static void main(String[] args) {
```

```
        Bank b = new Bank();
        b.openAccount();
    }
}
```

Problem 76: Object Created Inside Method

```
class Test {
    void create() {
        Test t = new Test();
        System.out.println("Object Created");
    }

    public static void main(String[] args) {
        new Test().create();
    }
}
```

🔹 **Problem 77: Method Calling Another Object's Method**

```
class A {
    void show() {
        System.out.println("Show Method");
    }
}

class B {
    public static void main(String[] args) {
        A a = new A();
        a.show();
```

```
    }
}
```

🔹 Problem 78: Constructor with Default Values

```
class Student {
    int id;
    String name;

    Student() {
        id = 0;
        name = "NA";
    }

    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s.id + " " + s.name);
    }
}
```

🔹 Problem 79: Overloaded Constructors – Different Data Types

```
class Demo {
    Demo(int x) {
        System.out.println("int constructor");
    }

    Demo(String s) {
        System.out.println("String constructor");
    }
```

```
    public static void main(String[] args) {
        new Demo(10);
        new Demo("Java");
    }
}
```

🔹 **Problem 80: Constructor vs Method Name**

```
class Test {
    Test() {
        System.out.println("Constructor");
    }

    void Test() {
        System.out.println("Method");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.Test();
    }
}
```

🔹 **Problem 81: Static Block + Constructor Order**

```
class Demo {
    static {
        System.out.println("Static Block");
    }
```

```
    Demo() {
        System.out.println("Constructor");
    }

    public static void main(String[] args) {
        new Demo();
    }
}
```

🔹 **Problem 82: Instance Block + Constructor Order**

```
class Demo {
    {
        System.out.println("Instance Block");
    }

    Demo() {
        System.out.println("Constructor");
    }

    public static void main(String[] args) {
        new Demo();
    }
}
```

🔹 **Problem 83: Static Variable Memory Sharing**

```
class Counter {
    static int count = 0;
```

```
  Counter() {
    count++;
  }

  public static void main(String[] args) {
    Counter c1 = new Counter();
    Counter c2 = new Counter();
    System.out.println(Counter.count);
  }
}
```

🔹 **Problem 84: Instance Variable Isolation**

```
class Demo {
  int x = 10;

  public static void main(String[] args) {
    Demo d1 = new Demo();
    Demo d2 = new Demo();

    d1.x = 99;
    System.out.println(d2.x);
  }
}
```

🔹 **Problem 85: this Keyword for Returning Object**

```
class Test {
  Test getRef() {
```

```
        return this;
    }

    public static void main(String[] args) {
        Test t = new Test();
        System.out.println(t == t.getRef());
    }
}
```

🔹 **Problem 86: Method Chaining with this**

```
class Demo {
    Demo a() {
        System.out.println("A");
        return this;
    }

    Demo b() {
        System.out.println("B");
        return this;
    }

    Demo c() {
        System.out.println("C");
        return this;
    }

    public static void main(String[] args) {
        new Demo().a().b().c();
```

```
  }
}
```

🔹 Problem 87: Method Overriding – Runtime Binding

```
class Parent {
  void show() {
    System.out.println("Parent");
  }
}

class Child extends Parent {
  void show() {
    System.out.println("Child");
  }

  public static void main(String[] args) {
    Parent p = new Child();
    p.show();
  }
}
```

🔹 Problem 88: Overriding vs Hiding (Static)

```
class A {
  static void show() {
    System.out.println("A");
  }
}
```

```
class B extends A {
  static void show() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

Output: A

🔹 Problem 89: super Keyword – Variable

```
class A {
  int x = 10;
}

class B extends A {
  int x = 20;

  void show() {
    System.out.println(super.x);
    System.out.println(this.x);
  }

  public static void main(String[] args) {
    new B().show();
  }
```

```
}
```

🔹 Problem 90: super Keyword – Constructor Chain

```
class A {
  A() {
    System.out.println("A");
  }
}

class B extends A {
  B() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    new B();
  }
}
```

🔹 Problem 91: Abstract Class Cannot be Instantiated

```
abstract class A {
  void show() {
    System.out.println("Hello");
  }
}

class B extends A {
  public static void main(String[] args) {
```

```
        A a = new B();
        a.show();
    }
}
```

🔹 Problem 92: Abstract Class with Concrete Method

```
abstract class Shape {
    abstract void draw();

    void info() {
        System.out.println("Shape Info");
    }
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }

    public static void main(String[] args) {
        Shape s = new Circle();
        s.info();
        s.draw();
    }
}
```

🔹 Problem 93: Interface Reference to Child Object

```
interface Vehicle {
```

```
  void run();
}

class Bike implements Vehicle {
  public void run() {
    System.out.println("Bike running");
  }

  public static void main(String[] args) {
    Vehicle v = new Bike();
    v.run();
  }
}
```

🔹 **Problem 94: Interface Variable is Implicitly Final**

```
interface Test {
  int x = 10;
}

class Demo {
  public static void main(String[] args) {
    System.out.println(Test.x);
  }
}
```

🔹 **Problem 95: Interface Multiple Implementation**

```
interface A {
  void a();
```

```
}

interface B {
  void b();
}

class C implements A, B {
  public void a() {
    System.out.println("A");
  }

  public void b() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    C c = new C();
    c.a();
    c.b();
  }
}
```

🔹 **Problem 96: Loose Coupling via Interface**

```
interface Message {
  void send();
}

class Email implements Message {
```

```
  public void send() {
    System.out.println("Email sent");
  }
}

class Test {
  public static void main(String[] args) {
    Message m = new Email();
    m.send();
  }
}
```

🔹 **Problem 97: Real-Time Example – Service Class**

```
class PaymentService {
  void pay() {
    System.out.println("Payment Done");
  }

  public static void main(String[] args) {
    PaymentService p = new PaymentService();
    p.pay();
  }
}
```

🔹 **Problem 98: Real-Time Example – Model Class**

```
class User {
  int id;
  String name;
```

```
    public static void main(String[] args) {
        User u = new User();
        u.id = 1;
        u.name = "Akash";
        System.out.println(u.id + " " + u.name);
    }
}
```

🔹 **Problem 99: Real-Time Example – Controller Style**

```
class Controller {
    void process() {
        System.out.println("Processing...");
    }

    public static void main(String[] args) {
        new Controller().process();
    }
}
```

🔹 **Problem 100: Final Level-1 Summary Program**

```
class Application {
    void start() {
        System.out.println("Application Started");
    }

    public static void main(String[] args) {
        Application app = new Application();
```

```
        app.start();
    }
}
```